1. CLA 可通过外设中断触发：

Task1-7 可以通过 ADC 或者 ePWM 来触发，如：
Task1：ADCINT1 或 EPWM1_INT；
Task2：ADCINT2 或 EPWM2_INT；
Task3：ADCINT3 或 EPWM3_INT；
Task4：ADCINT4 或 EPWM4_INT；
Task5：ADCINT5 或 EPWM5_INT；
Task6：ADCINT6 或 EPWM6_INT；
Task7：ADCINT7 或 EPWM7_INT；
Task8：ADCINT8 或 CPU Timer0；
为了使用 CLA，需要配置两块 RAM 区域，用于 CLA 与主 CPU 通讯使用，分别为 CPUtoCLARAM 以及 CLAtoCPURAM，其中 CPUtoCLARAM 区域数据 CPU 可读可写，CLA 只能读不能写，CLAtoCPURAM 区域数据 CLA 可读可写，CPU 只能读不能写。
CLA 可以直接访问 ePWM、HRPWM、比较器和 ADC 结果寄存器。

2. CMD 文件配置 CLARAM

与没有 CLA 的工程相比，带有 CLA 的工程需要为 CLA 配置一个 RAM 区域存放代码和数据。以工程为例：
CMD 开头定义了 CLA 暂存器大小用来暂存本地符号，不定义会警告，并且在 CMD 最后声明字段。并划分了一块 CLARAM1 给 CLA 存储数据，另外划分 CLA1_MSGRAMLOW 和 CLA1_MSGRAMHIGH 分别用于 CLA 与 CPU 的数据交互，这里 CLA 的程序是写在 flash 中，芯片上电后从 flash 搬运到 ram 中运行：

```
 8 _Cla1Prog_Start = _Cla1funcsRunStart;
 9 -heap 0x100
10 -stack 0x100
11
12 // Define a size for the CLA scratchpad area that will be used
13 // by the CLA compiler for local symbols and temps
14 // Also force references to the special symbols that mark the
15 // scratchpad are.
16 CLA_SCRATCHPAD_SIZE = 0x100;
17 --undef_sym=__cla_scratchpad_end
18 --undef_sym=__cla_scratchpad_start
19
```

```
PAGE 1 :   /* Data Memory */
           /* Memory (RAM/FLASH/OTP) blocks can be moved to PAGE0 for program allocation */
           /* Registers remain on PAGE1                                                  */

   RAMM0       : origin = 0x000100, length = 0x000300     /* on-chip RAM block M0 */
   RAMM1       : origin = 0x000400, length = 0x000400     /* on-chip RAM block M1 */
   RAML0       : origin = 0x008100, length = 0x000AFF     /* on-chip RAM block L0 */
/*   RAML1     : origin = 0x009000, length = 0x001000*/   /* on-chip RAM block L1 */
/*   RAMH0     : origin = 0x00A000, length = 0x002000  */ /* on-chip RAM block H0 */

   CLARAM1     : origin = 0x008c00, length = 0x000400

   CLA1_MSGRAMLOW   : origin = 0x001480, length = 0x000080
   CLA1_MSGRAMHIGH  : origin = 0x001500, length = 0x000080
```

```
   Cla1Prog             : LOAD = FLASHA,
                          RUN = RAML3,
                          LOAD_START(_Cla1funcsLoadStart),
                          LOAD_END(_Cla1funcsLoadEnd),
                          RUN_START(_Cla1funcsRunStart),
                          LOAD_SIZE(_Cla1funcsLoadSize),
                          PAGE = 0

   Cla1ToCpuMsgRAM      : > CLA1_MSGRAMLOW,   PAGE = 1
   CpuToCla1MsgRAM      : > CLA1_MSGRAMHIGH,  PAGE = 1
   Cla1DataRam0         : > CLARAM1,          PAGE = 1
   Cla1DataRam1         : > CLARAM1,          PAGE = 1
```

```
   CLAscratch          :
                         { *.obj(CLAscratch)
                         . += CLA_SCRATCHPAD_SIZE;
                         *.obj(CLAscratch_end) } > CLARAM1,
                         PAGE = 1
```

3. 主程序的配置

在主程序中，需要对 CLA 进行初始化工作。

首先设备初始化函数 SystemInit()中使能 CLA 的时钟：

```
   SysCtrlRegs.PCLKCR3.bit.CLA1ENCLK = 1;      // CLA1
```

对 CLA 进行初始化：CLAInit()

首先分配每个任务的向量地址，并且从 flash 中把程序搬运到 RAM 中运行。

```
   EALLOW;

   //Cla1Regs.MCTL.bit.SOFTRESET = 1;    //软件复位
/    MemCopy(&Cla1funcsLoadStart, &Cla1funcsLoadEnd, &Cla1funcsRunStart);
/    MemCopy(&Cla1mathTablesLoadStart, &Cla1mathTablesLoadEnd, &Cla1mathTablesRunStart);

   Cla1Regs.MVECT1 = ((Uint32)Cla1Task1 - (Uint32)&Cla1Prog_Start);
   Cla1Regs.MVECT2 = ((Uint32)Cla1Task2 - (Uint32)&Cla1Prog_Start);
   Cla1Regs.MVECT3 = ((Uint32)Cla1Task3 - (Uint32)&Cla1Prog_Start);
   Cla1Regs.MVECT4 = ((Uint32)Cla1Task4 - (Uint32)&Cla1Prog_Start);
   Cla1Regs.MVECT5 = ((Uint32)Cla1Task5 - (Uint32)&Cla1Prog_Start);
   Cla1Regs.MVECT6 = ((Uint32)Cla1Task6 - (Uint32)&Cla1Prog_Start);
   Cla1Regs.MVECT7 = ((Uint32)Cla1Task7 - (Uint32)&Cla1Prog_Start);
   Cla1Regs.MVECT8 = ((Uint32)Cla1Task8 - (Uint32)&Cla1Prog_Start);

   // Copy the CLA program code from its load address to the CLA program memory
   // Once done, assign the program memory to the CLA
   //
   // Make sure there are at least two SYSCLKOUT cycles between assigning
   // the memory to the CLA and when an interrupt comes in
   // Call this function even if Load and Run address is the same!
   MemCopy(&Cla1funcsLoadStart, &Cla1funcsLoadEnd, &Cla1funcsRunStart);
   MemCopy(&Cla1mathTablesLoadStart, &Cla1mathTablesLoadEnd, &Cla1mathTablesRunStart);
```

配置 CLA 程序和数据空间为 PROGE 和 RAM1E

```
  asm("   RPT #3 || NOP");
  Cla1Regs.MMEMCFG.bit.PROGE = 1;     // Configure the RAM as CLA program memory
  asm("   RPT #3 || NOP");
  //  Cla1Regs.MMEMCFG.bit.RAM0E = 1; // configure RAM L1, F28035 as CLA Data memory 0
  Cla1Regs.MMEMCFG.bit.RAM1E = 1;     // Configure RAM L2, F28035 as CLA data memory 1
  asm("   RPT #3 || NOP");
```

有些 CLA 的函数需要在程序启动后主动运行一次，比如数据的初始化工作，因此在配置 CLA 时，使能 CLA 应答 IACK，并且强行执行一次 CLA 任务。这里将 Task8 指定响应 M_INT2 中断，并使用 CLA1ForceTask2()函数执行一次 Task 任务并等待，完成 CLA 数据的初始化。

```
    Cla1Regs.MCTL.bit.IACKE = 1;

    //Enable Task 2 for initialization only
    Cla1Regs.MIER.all = M_INT2;

    // No need to wait, the task will finish by the time
    // we configure the ePWM and ADC modules

    Cla1ForceTask2andWait();
```

配置 CLA 任务触发函数，同样可以写在 CLA.c 文件中，该函数配置 TASK1 通过 ADCINT1 触发，当 ADC1 完成采样后，执行 Task1 程序进行数据处理，所以设置 Task1 的中断源为 ADCINT1，并使能中断响应，如需 PWM1INT 同理。

```
void CLATask_Configure(void)
{

    // Set Up CLA Task

    // Task 1 has the option to be started by either EPWM1_INT or ADCINT1
    // In this case we will allow EPMWM1INT to start CLA Task 1
    EALLOW;

    Cla1Regs.MPISRCSEL1.bit.PERINT1SEL = CLA_INT1_ADCINT1;  // 0=ADCINT1    1=none    2=EPWM1INT

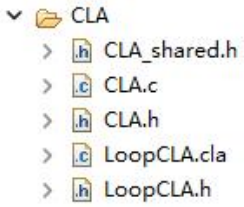
    // Configure the interrupt that woud occur each control cycles
    Cla1Regs.MIER.all = M_INT1;

    asm("   RPT #3 || NOP");

    EDIS;
}
```

4. CLA 主体程序

在工程中添加.cla 文件，claloop 程序在这里编辑，可直接赋值以下模板进行填充




```
#include "DSP28x_Project.h"
// Include the test header file whose name is based on the test name
// which is defined by the macro TEST on the command line
__interrupt void Cla1Task1 ( void )
{
}
__interrupt void Cla1Task2 ( void )
{
}
__interrupt void Cla1Task3 ( void )
{
}
__interrupt void Cla1Task4 ( void )
{
}
__interrupt void Cla1Task5 ( void )
{
}
__interrupt void Cla1Task6 ( void )
{
}
__interrupt void Cla1Task7 ( void )
{
}
```

```
__interrupt void Cla1Task8 ( void )
{
}
```

形式参考以下程序

```
interrupt void Cla1Task2 ( void )
{
    //mdebugstop();
    init_resolver_CLA();
    cycleCount = 100;
    cycleCount1 = 0;

    return;
}
//***************************************************************************
interrupt void Cla1Task4 ( void )
{
    __mdebugstop();      // CLA stop during debug session

    cycleCount1++;
    // *** Set up next sampling instant and update excitation sine wave ***
    rslvrOut.sineIndex = --rslvrOut.sineIndex & (rslvrIn.TABLE_LENGTH-1);  // DELAY_LENGTH must be 2^n
    //EPwm2Regs.CMPA.half.CMPA = sineTable[rslvrOut.sineIndex];

    // *** Sample sine and cosine feedbacks from ADC and remove their dc offset ***
    rslvrOut.sin_input = ((float)(AdcResult.ADCRESULT6)*(3.3/4096)) - rslvrIn.offsetS;
    rslvrOut.cos_input = ((float)(AdcResult.ADCRESULT6)*(3.3/4096)) - rslvrIn.offsetC;

    //resolver_algo_CLA();

    //EPwm1Regs.ETCLR.bit.INT = 1;          // clear EPwm1 INT flag
```

在.cla 文件开头要声明用到参数的分区，比如 rslvrin 是存在 CPUtoMsg 中的，即 CPU 可读可写，CLA 只读；rslvout 是存在 CLAtoMSGRAM，即 cla 可读写，CPU 只读。

```
#include "LoopCLA.h"
#include "DSP2803x_Adc.h"


// variables for cycle counting the execution times
Uint16 cycleCount,      // counter to store cycle count of selected code - debug purpose
       cycleCount1;     // counter to store cycle count of selected code - debug purpose


//=============================================================================
// Resolver algorithm library interface variables
//=============================================
// input to resolver loop
PFC_INPUT rslvrIn;
#pragma DATA_SECTION(rslvrIn, "CpuToCla1MsgRAM")

// output from resolver loop
PFC_OUTPUT rslvrOut;
#pragma DATA_SECTION(rslvrOut, "Cla1ToCpuMsgRAM")
```

.cla 文件中用到的参数和函数需要在响应的 h 文件中声明：

```
// ---------------------------------------------
void init_resolver_CLA(void);
int16 resolver_algo_CLA(void);


// ===========================================
// variables used between CPU and CLA
// ===========================================
extern int32 *sineTable;            // pointer to SINE_TABLE (CPU to CLA space)
extern float *firCoeff;             // pointer to FIR_COEFF_TABLE (CPU to CLA space)
extern PFC_INPUT PFCIn;      // variables in CPU to CLA space
extern PFC_OUTPUT PFCOut;    // variables in CLA to CPU space
```

共享参数需要在 CLA_share.h 中声明：

```
#include "IQmathLib.h"
#include "DSP2803x_EPwm_defines.h"
#include "DSP2803x_Cla_defines.h"


// *************************************************************************

#define XSTRINGIZE(s) STRINGIZE(s)
#define STRINGIZE(s) #s
#define XCONCAT(x,y) CONCAT(x,y)
#define CONCAT(x,y) x##y


// CLA C Tasks
// ============
interrupt void Cla1Task1();
interrupt void Cla1Task2();
interrupt void Cla1Task3();
interrupt void Cla1Task4();
interrupt void Cla1Task5();
interrupt void Cla1Task6();
interrupt void Cla1Task7();
interrupt void Cla1Task8();

```

附件为该工程 CLA 文件整合，仅供参考

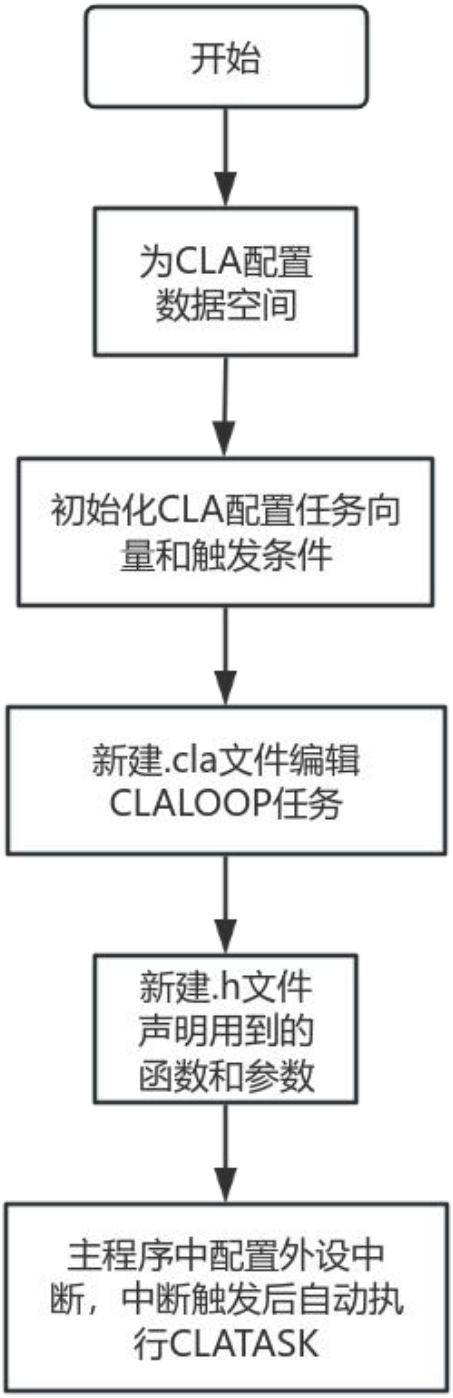


CLA.zip

Ti 的 CLA 程序 demo 如下：



Example_2803xCLaAdc.zip


开始
为CLA配置
数据空间
初始化CLA配置任务向
量和触发条件
新建.cla文件编辑
CLALOOP任务
新建.h文件
声明用到的
函数和参数
主程序中配置外设中
断，中断触发后自动执
行CLATASK